

Le mécanisme d'attention en vision

Des Transformers NLP aux Vision Transformers · Self-attention · ViT · Swin · BEiT/MAE

La rupture conceptuelle — CNN vs attention

L'attention est la réponse architecturale directe aux biais inductifs identifiés dans la section 2.1

Ce que les CNN ne peuvent pas faire

Champ réceptif limité : contexte global non accessible dès les premières couches

Invariance à la translation : position effacée alors qu'elle est porteuse de sens

Partage des paramètres : statistiques supposées stationnaires sur toute l'image

Réduction de résolution (pooling) : perte irréversible des détails fins

Ce que l'attention apporte

Champ réceptif global dès la 1ère couche : chaque token interagit avec tous les autres

Pondération contextuelle dynamique : calculée à la volée selon le contenu

Absence de biais de localité : patches éloignés $\leftrightarrow$ patches adjacents, même liberté

Encodage positionnel explicite : la position est un signal d'entrée, pas effacée

Pourquoi c'est crucial pour les manuscrits médiévaux

L'identification d'une lettre peut dépendre de mots situés plusieurs positions plus loin (ambiguïté des minimes, abréviations)

La mise en page est informative : lettrines, rubriques, réclames — leur position dans la page est le signal discriminant

L'attention encode exactement ce que fait un paléographe : regarder l'ensemble du mot (et de la phrase) pour décider

1. Genèse de l'attention

Du goulot d'étranglement RNN à la self-attention

Genèse du modèle Transformer

RNN (1990)

- ↳ LSTM / GRU (1997-2014) [résout le vanishing gradient]
- ↳ Encoder-Decoder (2014) [sépare lecture et génération]
- ↳ Attention (2015) [élimine le goulot d'étranglement]
- ↳ ConvS2S (2017)[explore la parallélisation]
- ↳ Transformer (2017) ★
[attention seule, entièrement parallèle]

L'attention de Bahdanau (2014) — du goulot d'étranglement au contexte dynamique

Le problème — goulot d'étranglement RNN (Sutskever, 2014)

Architecture encodeur-décodeur LSTM : toute la phrase source compressée dans un vecteur c de dimension fixe (256-512)

Pour les phrases longues : l'information sur les mots précoces est écrasée par les mots tardifs

Dégradation systématique de la qualité de traduction sur les longues phrases

La solution — vecteur de contexte dynamique

Au lieu d'un seul vecteur c , le décodeur calcule à chaque pas un contexte c_t :

$c_t = \sum_j \alpha_{tj} h_j$ (combinaison pondérée des états encodeur)

Poids $\alpha_{tj} = \text{softmax}(a(s_{t-1}, h_j))$ — réseau d'alignement appris de bout en bout

Pour produire « king » depuis « le roi » : α élevé sur l'état encodeur de « roi »

Propriétés clés de l'attention de Bahdanau

Alignement appris sans supervision explicite — le réseau découvre quels mots s'alignent

Accès direct à tous les états encodeur à chaque pas de décodage
Complexité $O(T^2)$ en la longueur de séquence — encore gérable pour des phrases

Précurseur direct de la self-attention : même principe, généralisé

Pertinence pour les manuscrits médiévaux

Modélise exactement le comportement d'un paléographe sur les minimes :

Pour déchiffrer *mnim*, le lecteur expert regarde l'ensemble du mot (contexte global) pour décider si c'est *minim*, *minum* ou *nimim*

L'attention alloue automatiquement des poids élevés aux positions contextuellement informatives

Self-attention — « Attention Is All You Need » (Vaswani et al., 2017)

*Supprimer complètement la récurrence · Faire de l'attention
le seul mécanisme de propagation*

La self-attention (intra-attention)

Chaque position d'une séquence peut interagir avec TOUTES les autres positions de la même séquence

Pas de récurrence → parallélisation complète de l'entraînement
Pas de limite de mémoire longue portée (contrairement aux LSTM)

Encodeur-Encodeur : chaque token enrichit sa représentation par le contexte

Décodeur-Encodeur : cross-attention (requêtes depuis le décodeur, clés/valeurs depuis l'encodeur)

Décodeur-Décodeur : self-attention causale (masquée pour ne regarder que le passé)

Les trois apports fondamentaux du Transformer

1. Self-attention globale

Champ réceptif infini dès la 1ère couche

$O(1)$ couches pour connecter deux positions quelconques

2. Parallélisation totale

Contrairement aux RNN séquentiels

Entraînement sur de très grands corpus possible

3. Architecture générique

Même mécanisme pour NLP, vision, audio, code

Foundation des modèles de langage (GPT, BERT) et de vision (ViT)

L'article le plus cité de l'histoire du deep learning

2. Le mécanisme de self-attention

Queries, Keys, Values — Scaled Dot-Product — Multi-Head Attention

Queries, Keys, Values et Scaled Dot-Product

Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q · K · V — l'analogie

Query (Q) : ce que le token courant cherche dans le contexte

Key (K) : ce que chaque token offre comme information

Value (V) : le contenu avec lequel chaque token contribue

$Q = X \cdot W_Q$ $K = X \cdot W_K$ $V = X \cdot W_V$
(projections linéaires apprises)

Les 4 étapes du calcul

1. Scores $S = QK^T \in \mathbb{R}^{n \times n}$ · $S_{ij} = q_i \cdot k_j$ = compatibilité token i avec token j
2. Mise à l'échelle $S / \sqrt{d_k}$ · Sans ça : pour d_k grand, variance élevée → softmax saturé → gradients nuls
Si $q, k \sim N(0,1)$: $\text{Var}(q \cdot k) = d_k$ → après $\div \sqrt{d_k}$: $\text{Var} = 1$
3. Softmax ligne par ligne $A_{ij} = \exp(S_{ij} / \sqrt{d_k}) / \sum_l \exp(S_{il} / \sqrt{d_k})$ → distribution sur tokens
4. Agrégation $\text{Out} = A \cdot V$ · Représentation de i = $\sum_j A_{ij} \cdot v_j$ (combinaison pondérée des valeurs)

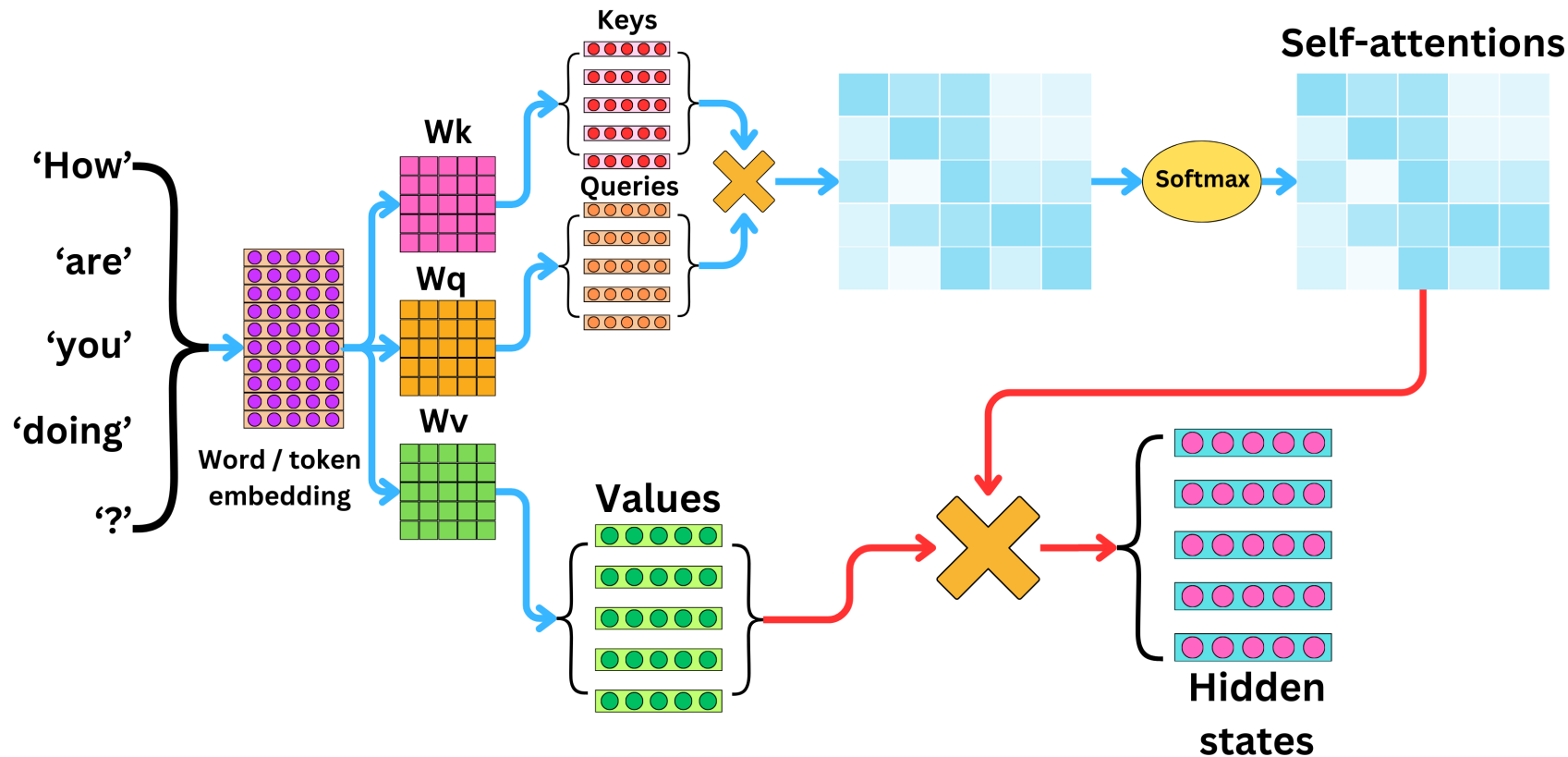
Interprétation sur les manuscrits médiévaux

Pour déchiffrer un jambage (Query), le modèle calcule des scores vers tous les autres tokens (Keys)

→ Tokens de jambages adjacents reçoivent des scores élevés (contexte immédiat pour minimiser)

→ Tokens de mots contextuels reçoivent aussi des scores élevés (contexte lexical longue portée pour lever l'ambiguïté)

La somme pondérée des Values intègre ce contexte multi-portée dans la représentation finale



Mécanisme d'attention

Complexité quadratique et Multi-Head Attention

Le coût de la globalité — et comment le démultiplier en parallèle

Complexité quadratique — le prix de la globalité

Matrice des scores $S = QK^T \in \mathbb{R}^{n \times n}$: $O(n^2 \cdot d_k)$ FLOPs et $O(n^2)$ mémoire

1024×1024 px, patches 16×16 → $n = 4096$ tokens

→ Matrice d'attention : $4096^2 \approx 16,8\text{M}$ entrées ≈ 64 Mo (float32, 1 tête)

→ ViT standard limité aux images basse résolution (224×224, 196 tokens)

→ C'est pourquoi le Swin Transformer existe (section suivante)

Multi-Head Attention (MHA)

h têtes d'attention parallèles, chacune avec ses propres W_Q , W_K , W_V

Dimension réduite par tête : $d_k = d_v = d_{\text{model}} / h$

$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$

$\text{head}_i = \text{Attention}(Q \cdot W_{Q_i}, K \cdot W_{K_i}, V \cdot W_{V_i})$

Paramètres totaux : $4 \cdot d_{\text{model}}^2$ (indépendant de h)

Tête 1 — Contexte immédiat

Aligne les jambages verticaux adjacents pour détecter les minimes

Attention concentrée sur les tokens voisins ($k=1-3$)

Tête 2 — Structures de ligature

Détecte les connexions entre lettres sur plusieurs positions

Attention étendue ($k=5-15$)

Tête 3 — Formules récurrentes

Identifie les formules de la langue médiévale

Contexte global — longue portée

3. Le Transformer complet

Bloc encodeur · Feedforward · Encodage positionnel

Bloc encodeur Transformer et encodage positionnel

Le bloc encodeur (architecture Pre-LN, préférée dans les ViT)

$x \rightarrow \text{LayerNorm} \rightarrow \text{MHA} \rightarrow (+x) \rightarrow \text{LayerNorm} \rightarrow \text{FFN} \rightarrow (+x) \rightarrow \text{sortie}$
(Les connexions résiduelles contournent chaque sous-module)

$$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$$

→ Traite chaque token indépendamment (pas d'interaction entre positions)

→ Dimensionnalité interne : $d_{\text{ff}} = 4 \cdot d_{\text{model}}$

→ Ajoute non-linéarité et capacité de transformation

Paramètres par bloc : $\text{MHA} (4 \cdot d^2) + \text{FFN} (8 \cdot d^2) + \text{LN} \approx 12 \cdot d_{\text{model}}^2$

ViT-B ($d=768$, 12 blocs) : $12 \times 12 \times 768^2 \approx 85\text{M}$ paramètres

Pre-LN (LN avant MHA/FFN) : plus stable qu'il n'entraîne, préféré dans tous les ViT modernes

Encodage sinusoïdal (Vaswani, 2017)

$$\text{PE}_{\text{pos}, 2i} = \sin(\text{pos} / 10000^{(2i/d)})$$

$$\text{PE}_{\text{pos}, 2i+1} = \cos(\text{pos} / 10000^{(2i/d)})$$

Déterministe · Généralise aux longueurs non vues

Encodage relatif implicite : $\text{PE}_{\text{pos}} \cdot \text{PE}_{(\text{pos}+k)}$ dépend de k uniquement

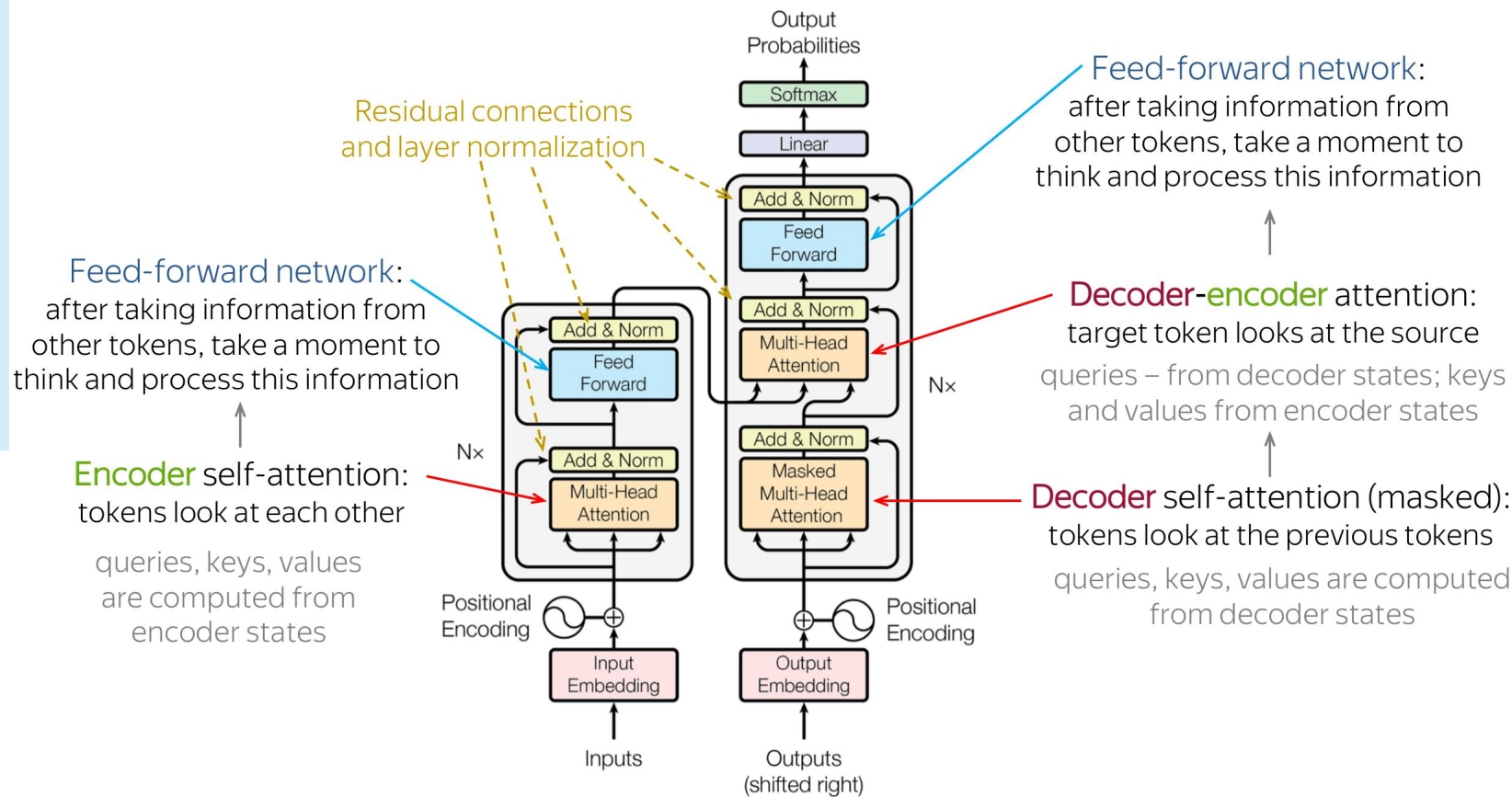
Encodage positionnel appris vs relatif

Appris : matrice $E_{\text{pos}} \in \mathbb{R}^{(n_{\text{max}} \times d)}$ — approche de ViT original

Inconvénient : ne généralise pas aux résolutions non vues
ViT 224×224 (196 patches) → image 448×448 (784 patches) : interpolation nécessaire

Relatif (Shaw, 2018 ; RoPE) : encode la distance entre positions directement dans les scores d'attention

→ Meilleure généralisation · adopté dans DeBERTa et ViT modernes



L'architecture typique d'un transformeur

4. Vision Transformer (ViT)

Une image vaut 16×16 mots — Dosovitskiy et al., ICLR 2021

ViT — Idée centrale et Patch Embedding

Traiter une image comme une séquence de tokens, exactement comme du texte en NLP

L'idée centrale (Dosovitskiy et al., 2020)

Observation : le Transformer NLP traite des séquences de tokens

Question : peut-on transformer une image en séquence de tokens ?

Réponse de ViT :

1. Découper l'image $H \times W$ en patches $P \times P$ non chevauchants
2. Aplatir chaque patch en un vecteur ($P^2 \cdot C$ dimensions)
3. Projeter dans l'espace de tokens (dimension d_{model})
4. Obtenir $n = (H/P) \times (W/P)$ tokens de dimension d_{model}
5. Appliquer le Transformer standard

Le patch de taille P est la brique élémentaire du ViT

Analogue au sous-mot (token BPE) en NLP

Patch Embedding — implémentation

Projection linéaire : $E \in \mathbb{R}^{(P^2 C \times d_{\text{model}})}$

$$z_i = x_i^{\text{patch}} \cdot E + e_{\text{pos},i}$$

Implémentée comme une convolution $P \times P$, stride P
(équivalent mathématiquement, plus efficace en mémoire)

Ajout de l'encodage positionnel appris $e_{\text{pos},i}$
→ La position du patch dans la grille est encodée explicitement

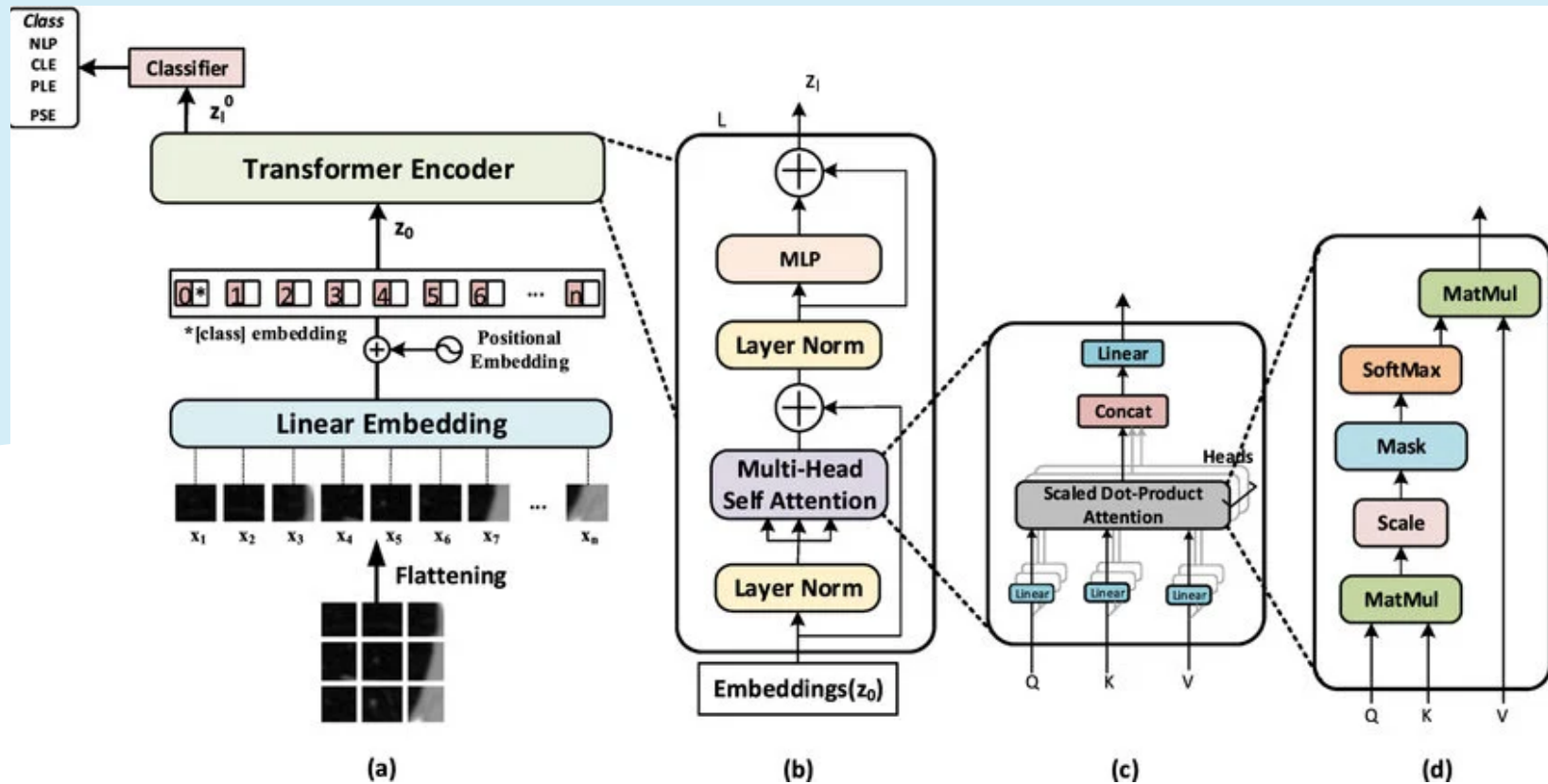
Choix de la taille de patch P pour les manuscrits

$P=32$: 49 tokens (224px) · rapide, moins détaillé

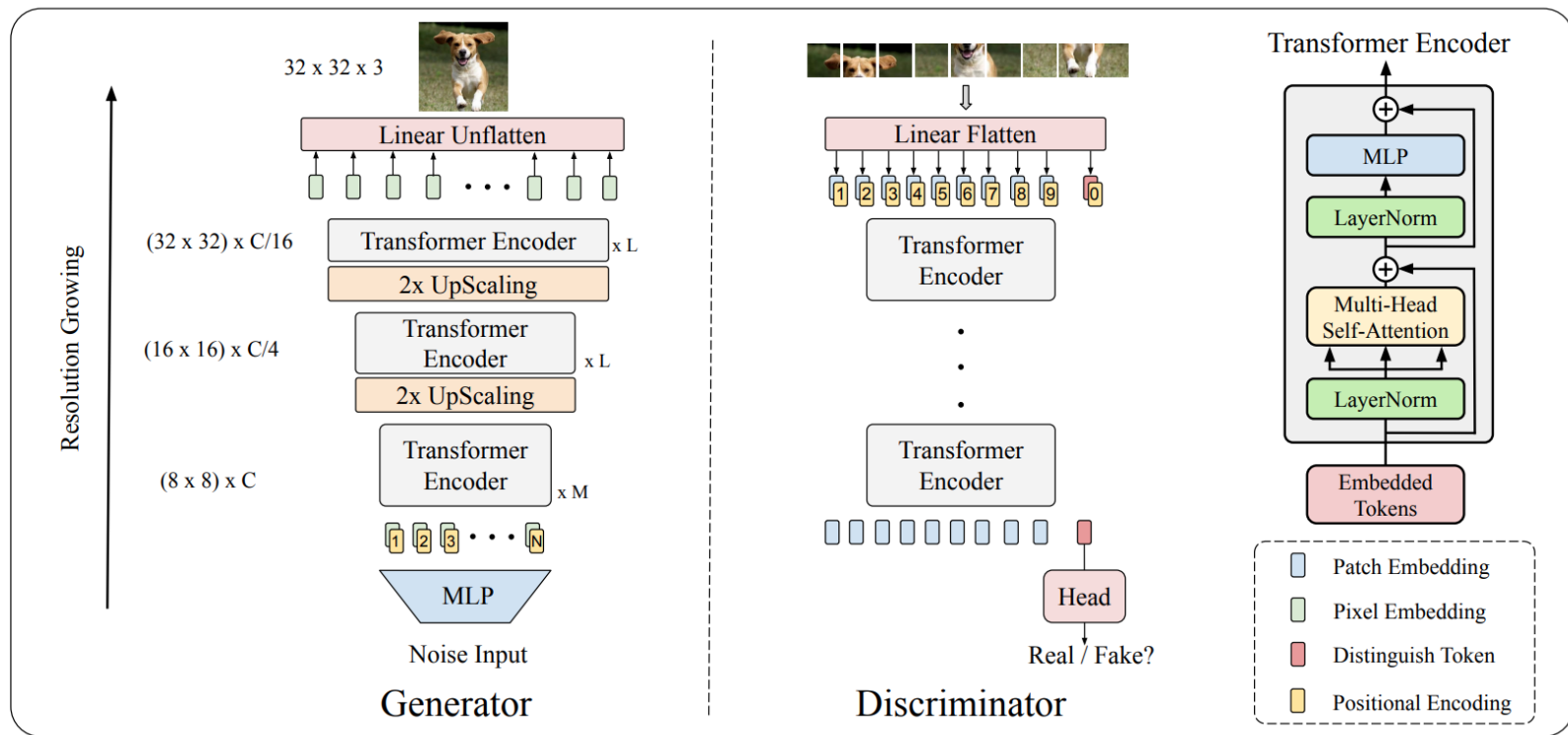
$P=16$: 196 tokens (224px) / 1024 tokens (512px) · compromis standard

$P=8$: 784 tokens (224px) · très précis, très lent

Pour les manuscrits : $P=16$ sur $512 \times 512 \rightarrow 1024$ tokens — praticable



Vision Transformer (2021)



TransGAN, une architecture générative basée sur les transformeurs

Token [CLS] et configurations standard de ViT

Le token de classification agrège l'information de tous les patches par attention

Token de classification [CLS]

Token synthétique appris préfixé à la séquence de patches :

$$Z = [z_{\text{cls}} ; z_1 ; \dots ; z_n] + E_{\text{pos}} \in \mathbb{R}^{((n+1) \times d)}$$

N'est associé à aucune région de l'image

À travers les L blocs Transformer, il agrège l'information globale par cross-attention avec tous les patches

Sa représentation en sortie z_o^{CLS} = représentation globale pour la classification

Alternative : mean-pooling de tous les tokens de patches en sortie (aussi efficace)

Modèle	Blocs	d_model	Têtes	Paramètres	ImageNet top-1
ViT-Ti/16	12	192	3	5,7M	~72%
ViT-S/16	12	384	6	22M	~79%
ViT-B/16	12	768	12	86M	~81%
ViT-L/16	24	1024	16	307M	~82%
ViT-H/14	32	1280	16	632M	~83%

5. Variantes pour les documents en haute résolution

DeiT · Swin Transformer · BEiT / MAE

DeiT et Swin Transformer

Efficacité en données · Complexité linéaire · Haute résolution

DeiT — Data-Efficient Image Transformers (Touvron, 2021)

Problème : ViT-B sur ImageNet seul → 77% (vs 81% avec JFT-300M)

Solution : distillation + augmentations agressives

Token de distillation z_{dist} ajouté à [CLS] :

$$L = (1-\lambda) \cdot L_{\text{cls}} + \lambda \cdot L_{\text{distill}}$$

Le token imite les prédictions d'un CNN enseignant (RegNetY-16GF)

Augmentations : RandAugment, CutMix, Mixup, Stochastic Depth

DeiT-B/16 : 81.8% top-1 sur ImageNet seul

→ Pour les manuscrits : fine-tuner avec peu de données annotées en utilisant DINOv2 comme enseignant

Swin Transformer — Attention à fenêtres décalées (Liu, 2021)

Problème : complexité $O(n^2)$ → inapplicable aux grandes images

Solution : attention locale dans des fenêtres $M \times M$ patches

→ Complexité linéaire en n : $O(4nd^2 + 2M^2nd)$

vs $O(4nd^2 + 2n^2d)$ pour MSA globale

Fenêtres décalées (SW-MSA) : décalage de $(M/2, M/2)$ couches alternées

→ Interactions cross-fenêtres sans surcoût

Architecture hiérarchique (4 stages) : feature maps à $H/4, H/8, H/16, H/32$

→ Compatible avec FPN, U-Net, Mask R-CNN

Swin-B + UperNet : mIoU 51.6 sur ADE20K (vs 45.9 ResNet-101)

→ Standard pour le layout analysis de pages de manuscrits

BEiT et MAE — pré-entraînement auto-supervisé par masquage

Apprendre des représentations riches sans annotation

· Adapté à n'importe quelle distribution d'images

BEiT — BERT Pre-training for Image Transformers (Bao, 2022)

Principe : reproduire BERT pour les images

→ Masquer ~40% des patches aléatoirement

→ Prédire leurs visual tokens (représentations discrètes d'un dVAE pré-entraîné)

$$L_{\text{BEiT}} = -E[\sum_{i \in M} \log p(z_i | x \setminus M)]$$

où M = indices masqués, z_i = visual token du patch i

BEiT est l'encodeur de TrOCR (Transformer pour l'OCR)

TrOCR = BEiT encodeur + GPT-2 décodeur → HTR state-of-the-art

Pour les manuscrits : fine-tuner BEiT pré-entraîné sur des scans améliore l'HTR même sans transcriptions

MAE — Masked Autoencoders (He et al., 2022)

Plus simple et scalable que BEiT :

→ Masquer 75% des patches (ratio élevé = tâche difficile)

→ Reconstruire les PIXELS (pas des tokens discrets)

→ L'encodeur ne voit que les 25% restants

$$L_{\text{MAE}} = (1/|M|) \sum_{i \in M} \|x_i - \hat{x}_i\|_2^2$$

Autoencodeur asymétrique :

Encodeur lourd (ViT) sur les patches visibles seulement

Décodeur léger (petit Transformer) reconstruit l'image entière

Intuition : reconstruire 75% depuis 25% = apprendre la structure sémantique

(impossible avec des features de bas niveau seulement)

→ Pré-entraîner MAE sur des scans non annotés → fine-tuner pour l'HTR

6. Visualisation et bilan

Grad-CAM · Attention Rollout · L'attention résout les limites des CNN

Grad-CAM vs Attention Rollout — interpréter CNN et ViT

Deux méthodes révèlent deux stratégies fondamentalement différentes pour lire un manuscrit

Grad-CAM (Selvaraju, 2017) — pour les CNN

Carte de chaleur depuis les gradients de la dernière couche convolutive

$\alpha^k_c = (1/Z) \sum_{ij} \partial y^c / \partial A_{ij}^k$ (poids = gradient moyen par canal)

$L^c = \text{ReLU}(\sum_k \alpha^k_c \cdot A^k)$ (ReLU = régions qui augmentent le score)

Sur un manuscrit (ResNet-50, classification de type) :

- Concentration sur les structures locales saillantes
- Lettrines ornées (fort contraste), rubriques rouges, bords de page
- Ne « voit » pas la structure globale du texte
- Lignes de texte ordinaires peu activées

Interprétation : ResNet classifie sur des features locales discriminantes

pas sur la structure globale de la mise en page

Attention Rollout (Abnar & Zuidema, 2020) — pour les ViT

Propagation récursive des matrices d'attention à travers toutes les couches

$\tilde{A}^l = 0.5 \cdot A^l + 0.5 \cdot I$ (mélange avec skip connection)

$\text{Rollout}^L = \tilde{A}^L \cdot \tilde{A}^{(L-1)} \cdot \dots \cdot \tilde{A}^1$

Ligne du token CLS → importances des patches

Sur un manuscrit (ViT-B/16, même tâche) :

- Attention distribuée sur l'ensemble des lignes de texte
- Sur-représentation des premières et dernières lignes (titres, souscriptions)
- Illustrations reçoivent de l'attention proportionnellement à leur taille

Sur une ligne HTR (TrOCR) :

- Chaque token attend ses voisins directs ET des tokens distants
- Tokens d'un jambage isolé regardent vers des lettres contextuellement contraignantes
- Résolution des ambiguïtés de minimes par contexte global

Pourquoi l'attention résout les limites des CNN

Limite CNN (section 2.1)	Solution apportée par l'attention	Architecture
Localité : contexte limité au champ réceptif	Champ réceptif global dès la 1ère couche — tous les patches interagissent en $O(1)$ couche	ViT / Swin
Invariance à la translation : position ignorée	Encodage positionnel explicite — la position est un signal d'entrée, pas effacée	ViT / Swin
Partage des paramètres : statistiques supposées stationnaires	Poids d'attention calculés dynamiquement selon le contenu — pas de partage spatial	ViT
Complexité quadratique en la résolution	Attention locale dans des fenêtres $M \times M$: complexité linéaire en n	Swin
Réduction de résolution destructive (pooling)	Architecture hiérarchique par patch merging avec skip connections — détails préservés	Swin
Distribution shift ImageNet → documents	BEiT/MAE : pré-entraînement auto-supervisé sur n'importe quelle distribution d'images	BEiT / MAE

Nuance : l'attention ne résout pas tout gratuitement — complexité quadratique réelle, moins performant que les CNN en faible données. Swin, ConvNeXt et les architectures hybrides combinent les deux paradigmes.

Récapitulatif — Architectures Vision Transformer

De la self-attention globale aux architectures adaptées aux documents haute résolution

ViT (2020)

Self-attention globale
 $n = (H/P) \times (W/P)$ tokens
Token [CLS] pour la classification
Encodage positionnel appris
Complexité $O(n^2)$

Limite : images basse résolution seulement (224×224)
Besoin de grandes données

DeiT (2021)

ViT entraînable sur ImageNet seul
Token de distillation
 $L = L_{\text{cls}} + \lambda \cdot L_{\text{distill}}$
Augmentations agressives

Pour les manuscrits :
Fine-tuning avec peu de données via distillation depuis DINOv2

Swin (2021)

Attention locale (fenêtres $M \times M$)
Fenêtres décalées → cross-window
Complexité linéaire en n
Hiérarchique (4 stages)

Pour les manuscrits :
Layout analysis haute résolution
Compatible FPN, U-Net, Mask R-CNN

BEiT / MAE (2022)

Pré-entraînement auto-supervisé
BEiT : prédire visual tokens (40% masqués)
MAE : reconstruire pixels (75% masqués)

Pour les manuscrits :
Pré-entraîner sur scans non annotés
BEiT → encodeur de TrOCR (HTR)
MAE → fine-tuning HTR avec peu de données